

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- **Java Is Robust**
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Java compiler can detect many problems that would first show up at execution time in other languages

Java has eliminated certain types of error-prone programming constructs found in other languages (default value, memory leak)

Java has a runtime exception-handling feature to provide programming support for robustness (nulpointerexp.)

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- **Java Is Secure**
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Java implements several security mechanisms to protect your system

- enforce array bound check(buffer overflow)
- not allow to perform unsafe operations such as pointer arithmetic or unchecked type casts
- not allow manual control over memory allocation and deallocation; users rely on automatic garbage collection
- Java Library provides APIs related to security, such as standard cryptographic algo's, Auth., and secure comm protocols.
- Exception handling

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Write once, run anywhere

With a Java Virtual Machine (JVM),
you can write one program that will
run on any platform

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- **Java Is Portable**
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

The execution of bytecode by the JVM makes java programs portable

Because Java is architecture neutral, Java programs are portable. They can be run on any platform without being recompiled

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- **Java's Performance**
 - **Java performance is slower than C**
 - **Can sometimes be even faster than compiled C code!**
 - **Incorporation of multithreading enhance the overall execution speed**
 - **Just-in-Time (JIT) can compile the byte code into machine code**
- Java Is Multithreaded
- Java Is Dynamic

JIT compiler turns Java bytecode into instructions that can be sent directly to a particular hardware platform's processor.

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- **Java Is Multithreaded**
- Java Is Dynamic

Multithread programming is smoothly integrated in Java,

whereas in other languages you have to call procedures specific to the operating system to enable multithreading

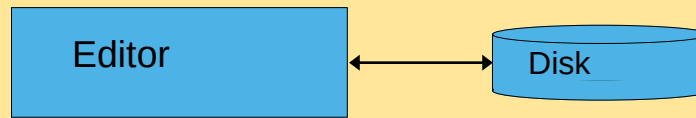
Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- **Java Is Dynamic**
 - Capable of dynamically linking a new class libraries, methods and objects.
 - Java can use efficient functions available in C/C++.
 - Installing new version of library automatically updates all programs

Basics of Java Environments

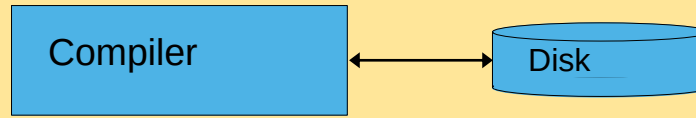
- Java programs normally undergo five phases
 - Edit
 - Programmer writes program (and stores program on disk)
 - Compile
 - Compiler creates *bytecode* from program
 - Load
 - Class loader stores bytecodes in primary memory
 - Verify
 - Verifier ensures bytecode do not violate security requirements
 - Execute
 - Interpreter translates bytecode into machine language

Phase 1



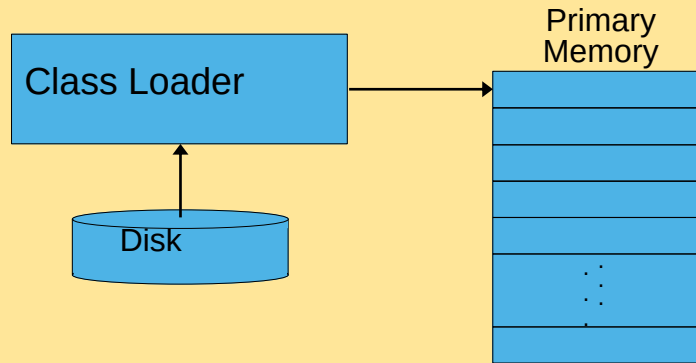
Program is created in an editor and stored on disk in a file ending with `.java`.

Phase 2



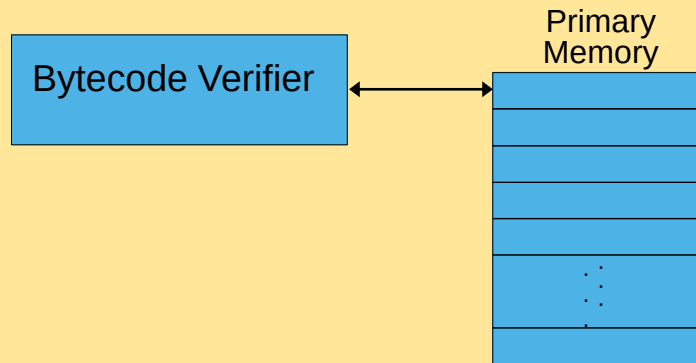
Compiler creates bytecodes and stores them on disk in a file ending with `.class`.

Phase 3



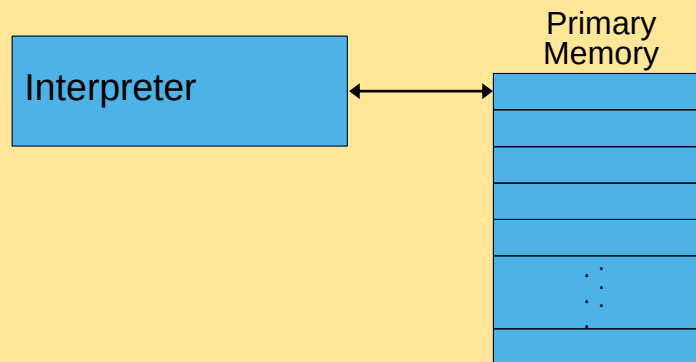
Class loader reads `.class` files containing bytecodes from disk and puts those bytecodes in memory.

Phase 4



Bytecode verifier confirms that all bytecodes are valid and do not violate Java's security restrictions.

Phase 5



Interpreter reads bytecodes and translates them into a language that the computer can understand, possibly storing data values as the program executes.

Java Environment

- Development tools-part of Java Development Kit (JDK)
- Classes and methods-part of Java Standard Library (JSL), also known as Application Programming Interface (API)

1. JDK:

- Appletviewer (for viewing applets)
- Javac (Compiler)
- Java (Interpreter)
- Javah (for C header files)
- Javadoc (for creating HTML description)

Java Environment

2. API

Contains hundreds of classes and methods grouped into several functional packages:

- **Language** Support Package
- **Utility** Packages (random num. gen., sys. date manipulation)
- **Input/Output** Packages
- **Networking** Packages (implementing networking app.)
- **AWT** Package (classes for painting graphics and images)
- **Applet** Package (web page using java)

The Evolution of Java

- Java 1.0 (96)
- Java 1.1 (97)(Add new library, redefine applet handling and reconfigured many features.)
- Java 2 (98)(Second generation). Version no:1.2 (Internal version number of java library). Also known as J2SE [Java 2 Platform Standard Edition].
 - Add swing, the collection framework, enhanced JVM etc.
- J2SE 1.3 (2000), J2SE 1.4 (2002), J2SE 5 (2004)
- J2SE 6 (2006), 7-(2011), 8(2014), 9(2017), ...
- Java SE 20 (Mar 2023), Java SE 21 (Sept. 2023),
- Java SE 22 (Mar 2024)

Comments

In Java, comments are preceded by two slashes (//) in a line, or enclosed between /* and */ in one or multiple lines

When the compiler sees //, it ignores all text after // in the same line

When it sees /*, it scans for the next */ and ignores any text between /* and */

Example

```
/* Traditional "Hello World!" program. */  
// package pack1;  
// import java.lang.System;  
class prog1  
{  
    public static void main (String args[])  
    {  
        System.out.println("Hello World!");  
    }  
}
```

Save program as **prog1.java**

Java Program Structure

Package Statement

- Javac command compiles the source code prog1.java then, generates **prog1.class** and store it under a directory which is called as name of the package
- package statement if used must be the first statement in a compilation unit. Its syntax is:

package packageName;

❑ For example:

package pack1;

Example

```
/* Traditional "Hello World!" program. */
```

```
// package pack1;
```

```
// import java.lang.System;
```

```
class prog1
```

```
{
```

```
    public static void main (String args[])
```

```
    {
```

```
        System.out.println("Hello World!");
```

```
    }
```

```
}
```

Import Statement

- The import statements are similar to #include statements in C and C++
- In the above program, System class of java.lang package is imported into all Java programs by default. The syntax of import statement is as:

```
import fullClassName;
```

- For example, the following import statement imports the System class from java.lang:

```
import java.lang.System;  
import java.lang.*;
```

Example

```
/* Traditional "Hello World!" program. */
```

```
// package pack1;
```

```
// import java.lang.System;
```

```
class prog1
```

```
{
```

```
    public static void main (String args[])
```

```
    {
```

```
        System.out.println("Hello World!");
```

```
    }
```

```
}
```

Classes

- Class declarations contain a keyword **class** and an identifier
- Class members are enclosed within braces. The syntax of defining a class is shown below:

```
class prog1  
    { // program code  
    }
```

Example

```
/* Traditional "Hello World!" program. */
```

```
// package pack1;
```

```
// import java.lang.System;
```

```
class prog1
```

```
{
```

```
public static void main (String args[])
```

```
{
```

```
    System.out.println("Hello World!");
```

```
}
```

```
}
```

Main method

- To execute a class, it must contain a valid main method
- It is the first method that automatically gets invoked when the program executed

```
public static void main (String args[])  
    { //instructions  
    }
```

Main method

```
public static void main (String args[])  
{  
    //instructions  
}
```

- The **main** method must always be defined as **public** to make it publicly accessible,
- **static** to declare it as a class member and
- **void** returns no value
- **args[]**: parameter is an array of class String. It provides access to command line parameters

Example

```
/* Traditional "Hello World!" program. */
```

```
// package pack1;
```

```
// import java.lang.System;
```

```
class prog1
```

```
{
```

```
    public static void main (String args[])
```

```
    {
```

```
        System.out.println("Hello World!");
```

```
    }
```

```
}
```

System class

```
System.out.println("Hello World!");
```

- invokes `println` method on object named `out` variable (of type `java.io.PrintStream`), which is a member of `System` class. The `println` method takes a String parameter and displays it on the console